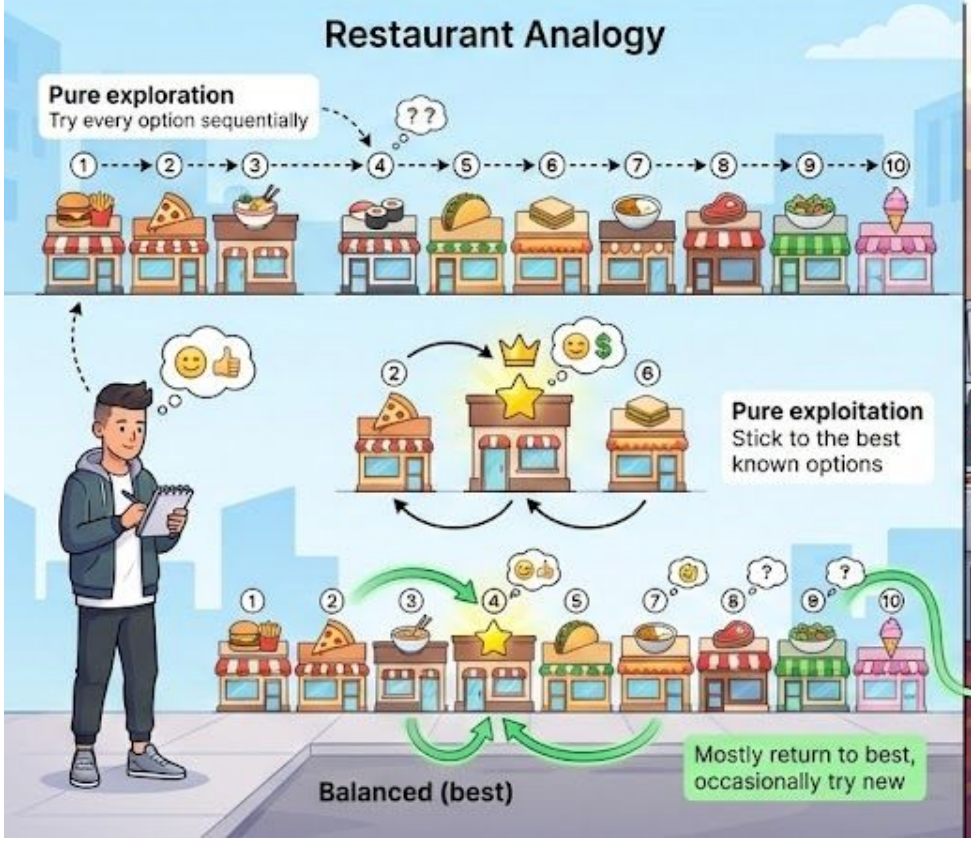
MULTI-ARMED BANDIT PROBLEM

“The Multi-Armed Bandit Problem”

Explore vs. Exploit: finding the best average reward.



THE MULTI-ARMED BANDIT PROBLEM

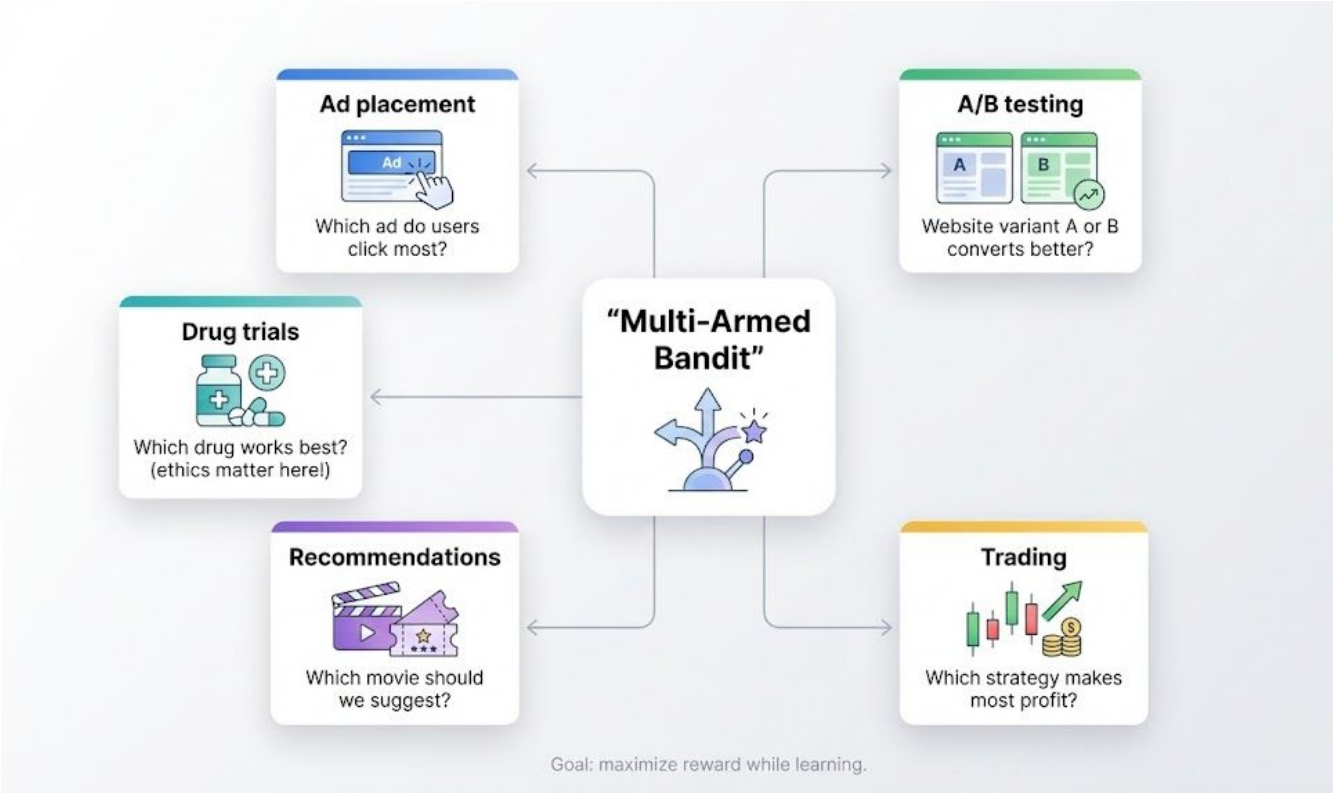
The Casino Scenario

- 10 free slot machines, each with different average payouts
- **Goal**: Identify which machine pays the most
- Each machine = “one-armed bandit” (lever steals your money = joke)

Formal Problem Statement

- n possible actions (pull n different levers/arms) – Each action a has unknown reward distribution – After k plays, receive reward R_k
- Challenge: Learn which arm is best

Why it matters?



ACTION-VALUE FUNCTIONS (Q- FUNCTIONS)

Definition

- Q_k(a) = expected reward for action a at play k – Equals average of all past rewards from action a

Mathematical Formula

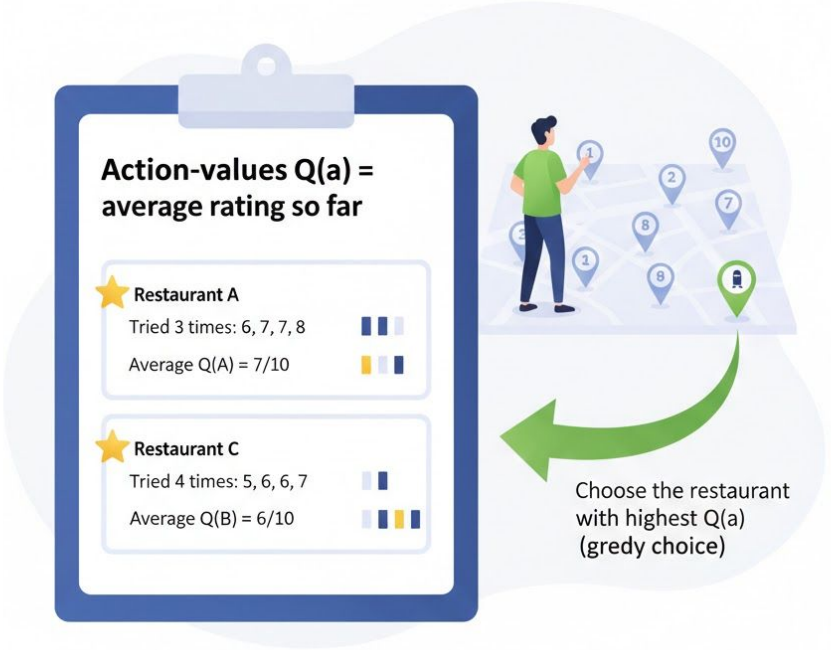
- Q_k(a)=(R_1+R_2+...+R_k)/count_a – where count_a = times we've taken action a

Key Insight

- Q(a) is called a value function: tells us "value" of taking action a – Also called action-value function (maps action → value)
- Abbreviated Q-function (Q = action-value in RL)

Usage

- Select best action: a* = argmax Q_k(a) over all actions



Formal Definition

- **Action-Value Function Q_k(a):** - Takes as input: an action a and play number k - Returns: expected reward for action a based on previous experience
  - Definition: arithmetic mean of all rewards received for action a
- **Math:** Q_k(a)=(R +R +...+R_k)/count_a
  - where: - R_i = reward at play i when taking action a - count_a = number of times we've taken action a - k = current play number
- **Example Calculation:** - Pull Arm 3 four times: get rewards [8, 9, 7, 9] - Q_4(Arm 3) = (8 + 9 + 7 + 9) / 4 = 33/4 = 8.25
- **Why It's Called "Action-Value":** - "Action" = we map FROM action (in- put) - "Value" = we map TO value (output) - Tells us the "value" (expected reward) of each action
- **Why Q Instead of V?** In reinforcement learning, Q stands for "Quality" of action-state pairs. Later you'll see: - V(s) = state-value function (value of being in state) - Q(s,a) = action-value function (value of taking action in state) - Deep Q-Learning = uses neural network as Q-function

Simple Algorithm Using Q

```
1. Initialize Q(a) = 0 for all actions
2. For each play:
   a. Choose action a* = argmax Q(a) (best known action)
   b. Pull arm a*, receive reward r
   c. Update: Q(a*) = (old_Q(a*) * count + r) / (count + 1)
   d. Increment count[a*]
```

The Greedy Approach: Always picking argmax Q(a) is “greedy” because you always exploit your best knowledge. Problem: never explores! If you got lucky with Arm 3 early on and it seemed great, you’d never try Arm 7 (which is actually better).

EXPLORATION VS EXPLOITATION

Exploration

- Trying different actions to learn their values
- Purpose: discover better actions than current best – Cost: receives lower reward in short-term
- Example: trying new restaurant might disappoint

Exploitation

- Using current knowledge to maximize immediate reward – Always choose best arm based on Q values
- Risk: miss undiscovered better options
- Example: always go to best known restaurant

The Dilemma

- No algorithm knows “when exploration is enough”
- Too little exploration → miss better arms
- Too much exploration → waste resources on bad arms
- **Balance is key**

Formal Definitions

Exploration:

- Acting in ways that generate new information about environment.
- Taking actions whose values are uncertain (high uncertainty).
- Goal: reduce uncertainty, learn true reward distributions.
- Cost: may receive lower immediate rewards - In the bandit: pulling arm you haven’t tried much

Exploitation:

- Using current knowledge to maximize reward.
- Taking actions with highest estimated value Q(a).
- Greedy behavior: always choose best.
- Goal: maximize cumulative reward now - In the bandit: pulling best arm according to current Q estimates

Why Balance Matters

Scenario 1 - Pure Exploration

- Pull each arm equally
- After 1000 plays with 10 arms: 100 pulls per arm
- You learn all distributions well
- But: consis- tently pulled bad arms!
- Total reward: maybe 5 * 1000 = 5000 - Inefficient use of plays

Scenario 2 - Pure Exploitation

- On first play, pull each arm once
- After 10 plays: best arm is Arm 3 (got 7)
- Pull Arm 3 for next 990 plays
- Total reward: 7 * 1000 = 7000 (better than exploration!)
- Problem: What if Arm 7 (best) was the 11th pull? You'd never try it

Scenario 3 - Balanced (Exploration/Exploitation)

- Mostly exploit best arm (Arm 3, average 9)
- Occasionally explore new arms (10% of time)
- Discover Arm 7 is also very good (average 8.5)
- Update and exploit mix of both - Total reward: ~8.5 * 1000 = 8500 (best of both!)

EPSILON-GREEDY STRATEGY

Core Idea

- Explore with probability (epsilon) – Exploit with probability 1 -

Algorithm

- Generate random number in [0, 1]
- If random < : choose random action (explore) – Otherwise: choose argmax Q(a) (exploit)

Typical Values

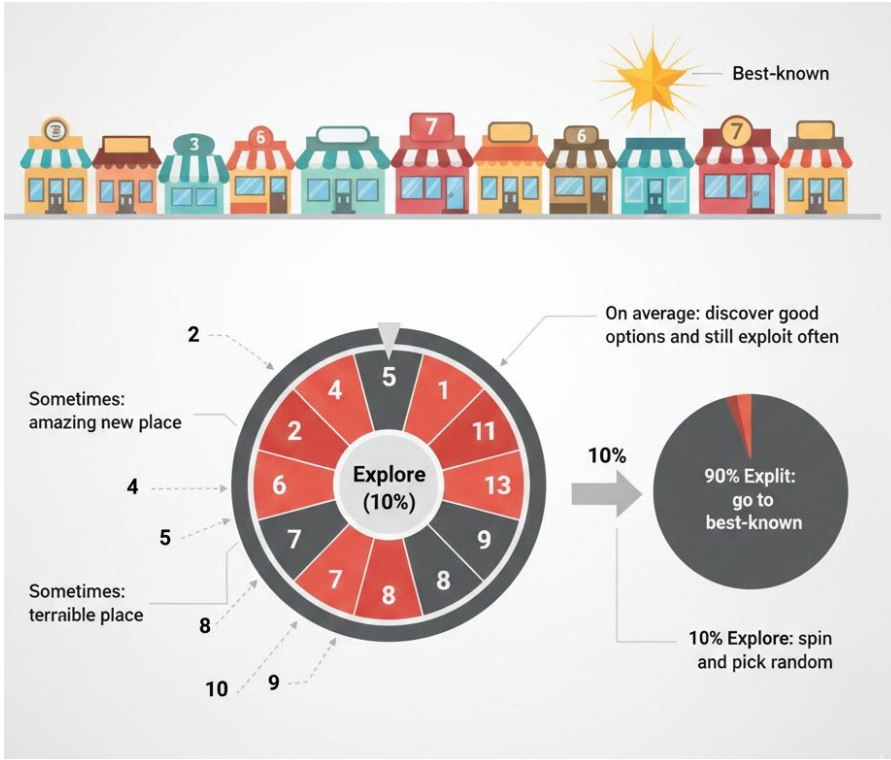
- = 0.1 to 0.2 (explore 10-20% of time)
- Can decay over time (explore more early, less late)

Pros

- Simple, intuitive, easy to implement

Cons

- Random exploration may pick terrible arm – No preference for uncertain arms



Algorithm

```
For each play:
  1. Generate random number r   [0, 1]
  2. If r <  :
     Choose action a randomly from uniform distribution
  3. Else:
     Choose a* = argmax Q(a)
  4. Execute action a, receive reward R
  5. Update Q(a) using reward
```

Example with epsilon = 0.1: - 90% of plays: exploit best arm - 10% of plays: randomly try any arm - Over 1000 plays: ~100 random, ~900 greedy

INCREMENTAL IMPLEMENTATION (ONLINE LEARNING)

Problem with Storing All Rewards

- Storing every reward for every action wastes memory – Recomputing mean each time is slow

Incremental Update Rule

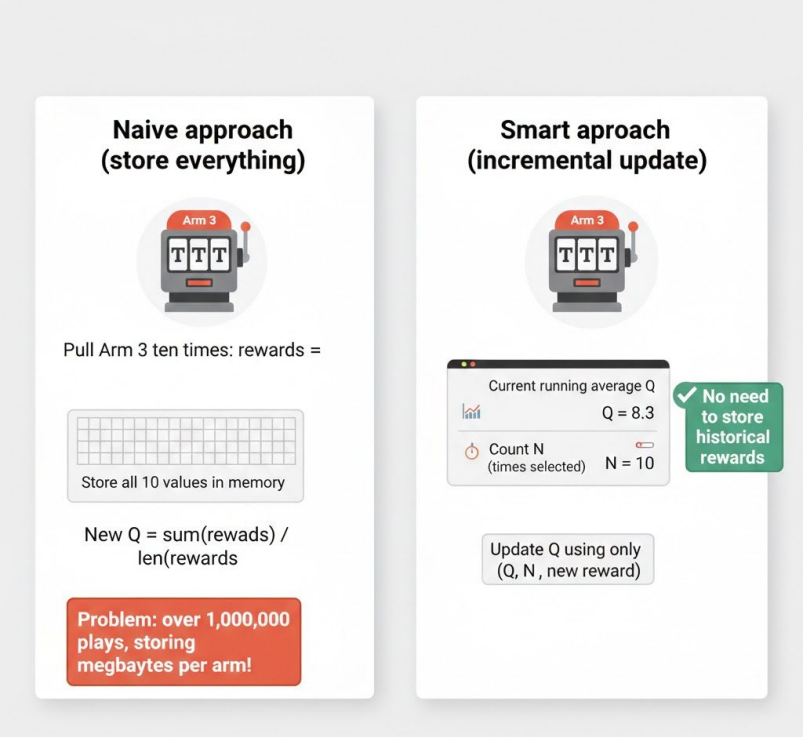
- Q_new = (Q_old * count + reward) / (count + 1) – OR equivalently:
- Q_new = Q_old + (reward - Q_old) / count

What We Store Per Action

- count_a = times action a was taken
- Q_a = current average reward estimate – 2 numbers per action (not all rewards!)

Example

- Pull arm, get reward 8
- Old: count=3, Q=7
- New: Q_new=(7*3+8)/4=29/4=7.25,count=4



TRACKING NON-STATIONARY PROBLEMS

Stationary Problem

- Reward distributions don’t change over time – Arm 3 always has mean 9
- Classical n-armed bandit

Non-Stationary Problem

- Reward distributions change over time
- “Concept drift” - best arm changes
- Example: Ad effectiveness changes with seasons

Problem with Simple Average

- Treats old and new rewards equally
- Doesn’t adapt to changing environment

Solution: Weighted Average (Exponential Recency)

- Q_new = Q_old + * (reward - Q_old)
  - (alpha) = learning rate, 0 < < 1
- Newer rewards weighted more heavily than old

Stationary vs. Non-Stationary

Stationary (Classical Bandit)

- Each arm has fixed underlying probability distribution
- Arm 3 always has p=0.9 of payout
- No matter when you pull it: same distribution - Average over all time is correct estimate

Non-Stationary (Real World)

- Reward distributions shift over time
- Ad effectiveness changes: summer vs winter
- Stock trading: market regimes change
- Website recommendations: user preferences evolve
- Medical treatments: new data → protocols change

Problem with Classical Average (Mathematically)

Scenario: Restaurant quality degraded

Visits over 10 years: 50 visits, average rating 8.5

But last week: 3 visits, ratings [6, 5, 6], average 5.7

Simple average says: Q = 8.5 (outdated!)

Intuition says:Q 5.7(recent,relevant)

Classical averaging treats all data as equally informative: $Q = (R + R + ... + R_n) / n$

This is fine for stationary (old data reliable), but terrible for non-stationary (old data irrelevant).

Solution 1: Simple Decay - Drop Old Data Only keep last k observations: Q = average of (R_{n-k+1}, R_{n-k+2}, ..., R_n)

Problem: arbitrary threshold, ignores all historical data

Solution 2: Exponential Recency Weighting Use learning rate (alpha) as a weight:

Standard form: Q_new = Q_old + * (reward - Q_old)

Rearranged form: Q_new = (1 -) * Q_old + * reward

where: - (0, 1) - Higher : recent rewards matter more (adaptive) - Lower : smooth out noise (stable)

Intuition: - (reward - Q_old) = "error" or surprise - scales this error - Multiply old by (1 -): discount old knowledge - Add * reward: incorporate new knowledge

Example with = 0.1:

```
Scenario: Restaurant has degraded
Q_old = 8.5 (estimate from long history)
New reward = 5.7 (recent visit)
  = 0.1

Q_new = 0.9 * 8.5 + 0.1 * 5.7
      = 7.65 + 0.57
      = 8.22

After many low rewards (5.7, 6, 5):
Q becomes ~6.5 (finally adapted!)
```

OPTIMISTIC INITIAL VALUES

The Problem with Zero Initialization

- Q(a) = 0 for all actions initially
- No incentive to explore early (all seem equally bad) – Agent doesn't distinguish explored vs. unexplored

Optimistic Initialization

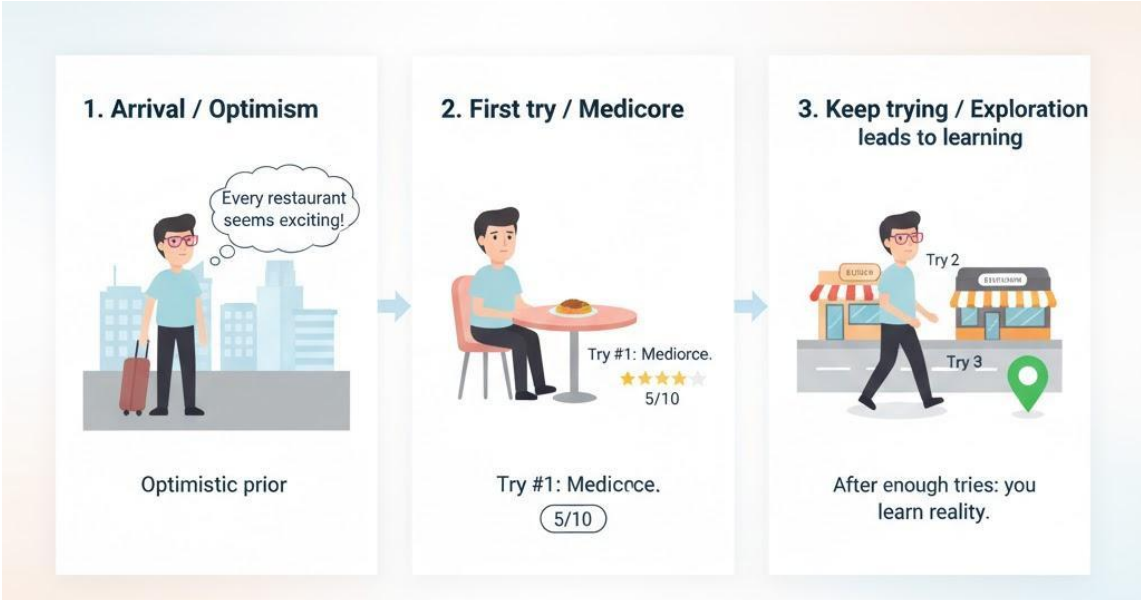
- Initialize Q(a) = high value (e.g., Q(a) = 5)
- Makes all actions look promising initially
- Agent naturally explores to confirm/refute optimism

How It Works

- First pull of arm: reward low (say 3)
- Update: Q(arm) = 0.1 * 3 + 0.9 * 5 = 4.8 (disappointing) – Pulls arm again to verify
- After many pulls: Q converges to true mean

Advantage

- Encourages early exploration naturally
- Simple, elegant solution



UPPER CONFIDENCE BOUND (UCB) ACTION SELECTION

Core Idea

- Select actions with highest upper confidence bound – Balance: high Q value AND high uncertainty

UCB Formula

- $A_t = \text{argmax} [Q(a) + c*\sqrt{(\ln(t) / N(a))}]$ – Q(a) = estimated value
- Second term = uncertainty bonus

Intuition

- High Q: good arm, select it
- High uncertainty: might be good, explore it – Low uncertainty: we know what it does, safer

Advantage over -greedy

- Intelligent exploration (not random) – Prioritizes uncertain promising arms

Quiz 1

In a k-armed bandit problem, what does $Q(a)$ represent during learning?

A The agent's current estimate of the expected reward for action a

B The number of times action a has been selected so far

C The true expected reward of action a, known to the agent

D The cumulative reward obtained from all actions except a

Quiz 2

An ϵ-greedy strategy is used in a stationary multi-armed bandit. What is the main role of the parameter ϵ in this algorithm?

A It determines how many actions exist in the bandit problem

B It controls how often the agent explores by choosing a non-greedy action

C It specifies the optimistic initial value assigned to each $Q(a)$

D It sets the learning rate in the value update rule for $Q(a)$

Quiz 3

In a basic bandit setting, what does $N(a)$ usually track for an arm a?

A The learning rate used when updating $Q(a)$ for arm a

B The total reward received from all arms combined

C The estimated average reward of arm a

D How many times arm a has been selected so far

Quiz 4

What is the key idea behind the exploration–exploitation trade-off in multi-armed bandits?

A Always choose the arm with the highest $Q(a)$ so far and never try other arms

B Ignore past rewards and only use the most recent reward when choosing arms

C Always choose arms at random so that all are explored equally

D Sometimes try uncertain arms to gather information, and sometimes use the best-known arm to gain reward